

Computer Programming Assignment 2024

De Santis Lorenzo - 3163450

Task 1: Compute the `cost()` and `delta_cost()` method.

The computation of the `cost()` method in the `KSAT.py` file exploits both vectorization and broadcasting to calculate the cost, it is expressed as:

$$c = \text{np.sum}(\text{np.prod}((1-s*x[\text{index}])/2, \text{axis}=1))$$

This expression aligns directly with the mathematical formulation of the cost function:

$$E(\hat{x}) = \sum_{m=1}^M \left(\prod_{k=1}^K \frac{1 - s_{mk} \hat{x}_{mk}}{2} \right)$$

The `delta_cost()` method implemented in the `KSAT.py` file calculates the change in the cost function (Δcost) when flipping a variable. It focuses only on the clauses affected by the flip leveraging the “clauses” array, checking their satisfaction before and after the change.

The cost increases if a clause becomes unsatisfied and decreases if it becomes satisfied, ensuring efficient computation by avoiding recalculating the entire cost.

A slightly more efficient version of `delta_cost()` is proposed which becomes more efficient for large K .

Task 2.1: investigate the evolution of the acceptance rate during optimisation: fix $N = 200$ and $M = 200$ and choose proper values for `mcmc_steps`, `steps`, `beta0` and `beta1` parameters to approach the lowest possible cost.

In the `KSATrun.py` file the chosen seed for the `KSAT.KSAT` method is 3163450, and for the `Simann.simann` method is 5436130.

In order to achieve a balance between computational efficiency and the accuracy of the solution, the parameters of the Simulated Annealing algorithm were selected as follows:

- `mcmc_steps` = 5000
- `anneal_steps` = 50
- `beta0` = 0.01
- `beta1` = 5.0

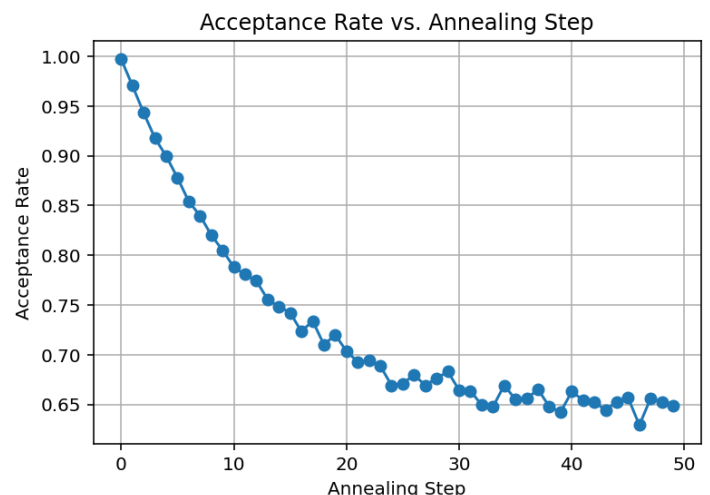


Figure 1

`mcmc_steps` and `anneal_steps` were chosen based on a grid search approach to identify the optimal configuration that minimises computational time while maintaining a high probability of finding a solution. Specifically, values for `mcmc_steps` were tested up to 10,000 and `anneal_steps` up to 200. The chosen values represent a good compromise between computational cost and performance.

The choice of `beta0` and `beta1` was guided by the need to ensure a smooth annealing process. `beta0` was set to a low value (0.01) to ensure a very high, finite, starting temperature, which facilitates exploration of the solution space by allowing a high initial acceptance rate.

On the other hand, `beta1` was set relatively low (5.0) to allow sufficient cooling and to ensure that the algorithm converges toward optimal solutions in the later stages of the annealing schedule. The relatively low value of `beta1` (5.0) was chosen in combination with a relatively high number of `anneal_steps` (50). This combination ensures good mixing, as shown in Figure 1, allowing the algorithm to maintain a sufficiently high acceptance rate during the cooling process.

The graph presented (Figure 1) illustrates the evolution of the acceptance rate during the annealing process. As expected, the acceptance rate starts at approximately 1.0 and decreases smoothly as the temperature decreases, stabilising around 65% at the end of the annealing process. This behavior confirms that the chosen parameters effectively balance exploration and exploitation during the optimisation process.

Task 2.2: *is Simulated Annealing always able to find a solution to 3 - SAT ? Repeat the analysis for a larger number of clauses, i.e. $M \in \{400, 600, 800, 1000\}$. Is Simulated Annealing always able to find a solution? What is happening in the optimisation problem?*

No, Simulated Annealing is not always able to find a solution for 3-SAT problem.

As the number of clauses (M) increases for a fixed number of variables (N) the number of solved instances decreases. This can be attributed to the increasing complexity and constraints imposed by the larger number of clauses. Which may result in cost landscape with many local minima.

For smaller values of M (M=400), Simulated Annealing has a higher probability for finding a solution, as the problem is relatively less constrained, given the large number of variables (N = 200) there are many feasible configurations and many solutions are likely to satisfy all the clauses.

As M increases (M = 800 or M = 1000) the algorithm often ends up with a cost greater than 0, indicating that the 3-SAT instance has not been solved.

This occurs because the problem becomes increasingly over-constrained: a higher number of clauses imposes stricter logical constraints on the variables, significantly shrinking the space of feasible solutions.

The ratio M/N is a critical factor in determining the difficulty of K-SAT problems, particularly for 3-SAT. Empirical studies show that 3-SAT exhibits a phase transition behavior, with a critical threshold estimated to be around 4.25. Below this threshold, most instances of the problem are satisfiable, and solutions can be found more readily. However, as M/N approaches and exceeds 4.25, the problem transitions to being largely unsatisfiable, and finding solutions becomes exceedingly difficult. This further explains why the performance of Simulated Annealing declines as M increases.

What is happening in the optimisation problem as M increases is that with larger M , problem's constraint density increases, meaning that each variable is involved in more clauses, hence it is more difficult to find an assignment of the variables capable of satisfying all clauses in which the variable is present simultaneously.

From an analytical perspective, the cost function $E(\hat{x})$ becomes significantly more complex with larger M , resulting in a landscape with a greater number of local minima. These local minima can trap the Simulated Annealing algorithm, making it challenging to reach the global minimum corresponding to a satisfying assignment.

Furthermore, as the system cools (i.e., as β , the inverse temperature, becomes large), the algorithm becomes increasingly selective in accepting worse solutions. This reduced acceptance of uphill moves limits its ability to escape local minima. If no satisfying assignment exists, the cost function stabilises at a non-zero value, indicating that the algorithm has plateaued and failed to find a solution.

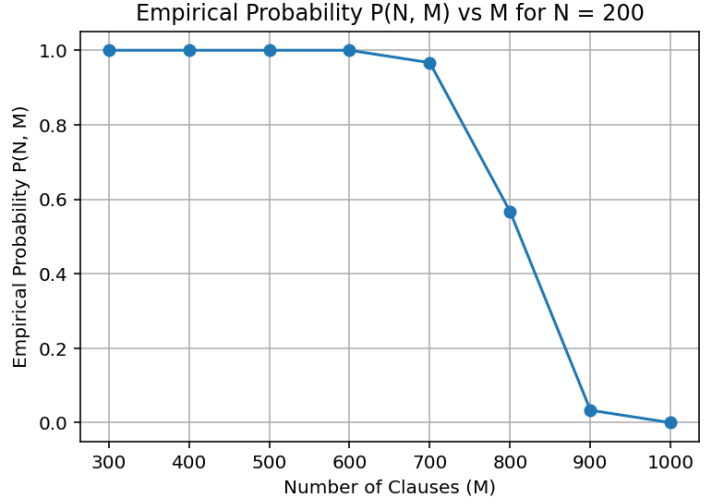
Task 3: *Compute the empirical probability of solving a random instance, at fixed N and M : $P(M,N) = \# \text{ of solved problem instances} / \# \text{ total problem instances}$, fix $n = \# \text{ total instances} = 30$.*

The code for this problem is in the 3-SAT_properies file contained in the submission folder.

Nb: the provided graphs are computed using 30 instances to have a good sample but for computational reasons the submitted code has 10 iterations.

Task 3.1: *How does $P(N,M)$ change by keeping fixed N and increasing M ? Fix $N = 200$ and plot $P(N,M)$ vs M for different values for the number of clauses, (e.g., $M \in \{400,500,600,700,800,900,1000\}$). Try to identify the so-called algorithmic threshold $M_{Alg}(N = 200)$, i.e. the number of clauses at which the empirical probability of solving a clause with Simulated Annealing is equal to $1/2$*

Figure 2



As M increases while keeping $N = 200$ fixed, $P(N, M)$, the empirical probability of solving a 3-SAT instance using Simulated Annealing, decreases.

For smaller values of M (e.g., $M = 300, 400, 500$), where the problem is under-constrained, the algorithm almost always finds a solution, resulting in $P(N, M) \approx 1$.

The algorithm demonstrates remarkable efficiency for values of M up to 700, which is surprising given the increasing complexity of the problem as M grows. Despite this rise in complexity, the probability of solving a 3-SAT instance remains close to 1, highlighting the robustness of the Simulated Annealing algorithm within this range.

Starting from $M = 700$, however, the empirical probability begins to decline significantly. It drops from nearly 1 at $M = 700$ to approximately 0.1 at $M = 900$, and eventually reaches 0 at $M = 1000$. This sharp decrease underscores the increasing difficulty for the algorithm to find a solution as M becomes large and the problem becomes over-constrained.

This behavior aligns with theoretical expectations, as the space of feasible solutions diminishes with higher M , resulting in a more challenging search landscape. The result that fine-tuning algorithm parameters could improve performance, albeit at the cost of increased computational complexity.

We can identify the algorithmic threshold $M_{\text{Alg}} (N = 200)$ in a value slightly greater than $M = 800$, specifically $M_{\text{Alg}} (N = 200) \approx 810$, where the empirical probability of solving a 3-SAT instance with Simulated Annealing reaches approximately $1/2$.

As discussed earlier in this report, the M/N ratio plays a crucial role in determining the difficulty of solving 3-SAT problems.

The observed value of $M_{\text{Alg}} \approx 810$ is remarkably close to the theoretical optimal ratio M/N for 3-SAT problems, which is estimated to be approximately 4.25. In this case, the calculated ratio is $M/N = 810/200 = 4.05$, deviating only slightly from the theoretical value. This result highlights that, even with the current parameter settings, the Simulated Annealing algorithm performs highly effectively, approaching the theoretical limit without requiring additional optimisation.

Task 3.2: What is the form of the probability $P(M, N)$ close $M_Alg(N)$? What is happening to the system of variables and clauses when you are approaching the algorithmic threshold?

The probability $P(N, M)$ decreases sharply as M approaches the algorithmic threshold $M_Alg(N) \approx 810$. Specifically, $P(N, M)$ transitions from values close to 1 (high probability of solving) to nearly 0 (low probability of solving) within a narrow range of M . This sharp decline around the algorithmic threshold resembles the shape of a sigmoid curve, which is characteristic of the phase transition behavior in 3-SAT problems.

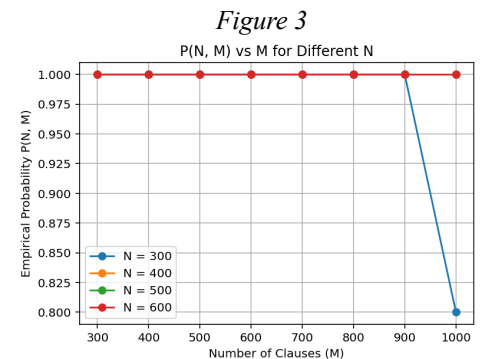
As M approaches the algorithmic threshold, the following occurs:

- **Shrinking of the Solution Space:** Since the number of variables is fixed ($N = 200$), increasing M leads to a higher number of logical constraints. Each variable becomes involved in more clauses, which significantly increases the complexity of the problem and rapidly shrinks the solution space.
- **Increase in Function Complexity:** As M increases, the cost function becomes increasingly rugged, with a growing number of local minima. This heightened complexity makes it more difficult for the algorithm to escape local minima and find a global solution, further challenging its ability to solve the instance.

The sigmoid-like curve reflects the probabilistic nature of the phase transition, where the system transitions from a satisfiable regime ($P(N, M) \approx 1$) to an unsatisfiable regime ($P(N, M) \approx 0$) over a relatively narrow range of M . This behavior highlights the critical role of constraint density and cost function ruggedness in driving the phase transition dynamics of 3-SAT problems.

Task 3.3: Does the algorithmic threshold depend on the choice of N ? Consider different values of the number of variables, e.g. $N \in \{300, 400, 500, 600\}$ and repeat the previous analysis, plotting $P(N, M)$ vs M and identifying M_Alg . Can you understand the scaling of M_Alg with N ? If you find the proper rescaling (of M), you should be able to approximately collapse the curves of $P(N, M)$ obtained with different values of N .

Nb: To tackle this exercise and obtain meaningful results, the parameters were deliberately worsened, as the algorithm already exhibited optimal behavior for $N = 200$ with respect to increasing M . Increasing the number of variables N in this context, where the algorithm is very efficient, has the effect of further raising the threshold M/N at which the transition between solved and unsolved problems occurs.



As a result, the probability $P(N, M)$ remains consistently close to 1 for $N > 300$, even for instances with $M = 800$ or $M = 900$, producing graphs similar to *Figure 3*.

After setting the following parameters:

- `mcmc_steps` = 3000
- `anneal_steps` = 20
- `beta0` = 0.01
- `beta1` = 4.0

It is expected a deterioration in the algorithm's performance. Specifically, we anticipate the value of $M_Alg(N)$ in the graph of the collapsed curve to be less than 4.25 (the optimal value).

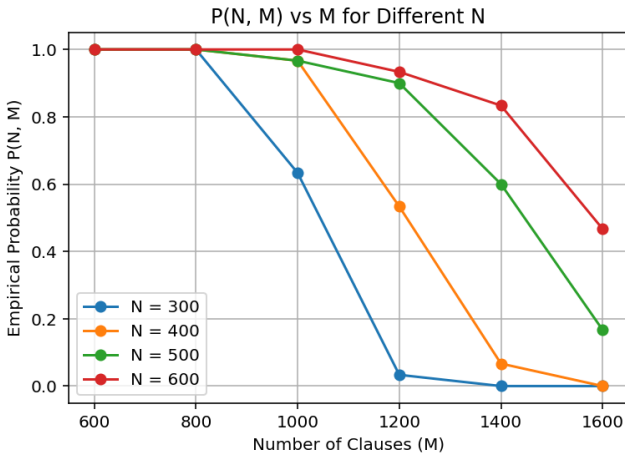


Figure 4

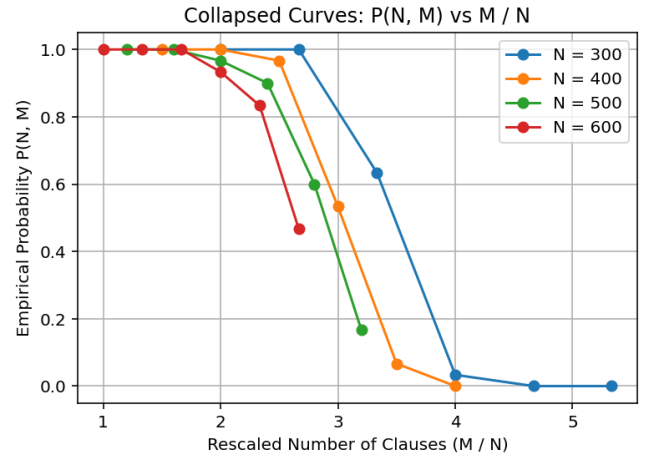


Figure 5

Figure 4 shows the empirical probability $P(N, M)$ as a function of M for different N . $P(N, M)$ drops to approximately 1/2, clearly depends on N . As N increases, M_Alg also increases, indicating that the number of clauses required to reach the critical transition grows with the problem size.

This dependency arises because a larger number of variables introduces more variability in the clauses, making the problem less constrained and easier to solve with fewer conflicts. Consequently, more clauses are needed to reach the critical point where satisfiability probability drops.

The curves in Figure 4 systematically shift to the right as N increases, demonstrating that larger M values are required for higher N to observe the same transition behavior. However, the shape of the transition remains consistent, confirming that the algorithmic threshold scales proportionally with N . This suggests that the relationship between M_Alg and N is approximately linear.

Figure 5 illustrates the relationship between the empirical probability $P(N, M)$ and the rescaled variable M/N , for various values of N . The purpose of the figure is to assess whether rescaling M by N successfully collapses the curves into a single

universal one, thereby demonstrating that the statistical behavior of the system is static and well-captured by the parameter M/N .

Although previous analyses identified a critical transition value near $M/N \approx 4.25$, the necessary tradeoff between parameter optimisation and computational time has affected the precision of this estimation, confirming the initial expectations.

Consequently, the transition from $P > 1/2$ (most formulas satisfiable) to $P < 1/2$ (most formulas unsatisfiable) is now observed, on average, at approximately $M/N \approx 3$, based on the central tendency of the curves in the figure.

This shift in the observed critical value underscores the challenges posed by computational limitations when conducting large-scale empirical analyses. Despite this discrepancy, the general behavior of the system is well-represented, with M/N providing a reliable rescaling metric. The results highlight a sharp phase transition around the critical M/N , which becomes increasingly well-defined as N grows, consistent with statistical scaling phenomena.

Final Remarks

If the algorithm had been run with the original parameters, the results would have been consistent with the findings in the first part of the report, confirming that the threshold lies around $M/N \approx 4.25$. The observed deviation in the collapsed curve's threshold can be attributed to adjustments in the parameters, which were necessary to balance computational feasibility with the scope of the analysis. This tradeoff impacted the precision of the results but does not contradict the theoretical expectation or the findings from prior simulations.